EXPERIMENT - 5

AIM:

Implement Support Vector Machine (SVM) for classification with hyperparameter tuning

1. Dataset Source

Dataset Name: Iris Flower Dataset
File Used: Pima Indians Diabetes Dataset

2. Dataset Description

This dataset is used to predict whether or not a patient has diabetes based on diagnostic measurements. All patients here are females at least 21 years old of Pima Indian heritage.

- **Type:** Numerical / Multivariate
- **Domain:** Healthcare / Medicine
- **Target Variable:** Outcome (0: Non-Diabetic, 1: Diabetic)
- **Key Features:**

Feature	Description	Data Type
Pregnancies	Number of times pregnant	Numerical
Glucose	Plasma glucose concentration (2 hours in an oral glucose tolerance test)	Numerical
BloodPressure	Diastolic blood pressure (mm Hg)	Numerical
SkinThickness	Triceps skin fold thickness (mm)	Numerical

Insulin	2-Hour serum insulin (mu U/ml)	Numerical
BMI	Body mass index (weight in $kg/(height\ in\ m)^2$)	Numerical
DiabetesPedigreeFunction	A function which scores likelihood of diabetes based on family history	Numerical
Age	Age in years	Numerical
Outcome	Class variable (0 if non-diabetic, 1 if diabetic)	Target (Label)

3.Mathematical Formulation of the Algorithm

Support Vector Machine (SVM)

SVM finds the optimal hyperplane that maximizes the **margin** between two classes.

- **Hyperplane Equation:** w . x + b = 0
- **Margin:** The distance between the hyperplane and the nearest data points (Support Vectors). The goal is to maximize 2/||w||
- **Kernel Trick:** For non-linear data, SVM uses a kernel function $K(x_i, x_j)$ to project data into higher-dimensional space where it becomes linearly separable.

$$\boxed{\textit{RBF Kernel: } K(x, y) = \exp(-\gamma \|x - y\|^2)}$$

4. Algorithm Limitations

- **Sensitivity to Scaling:** SVM calculates distances; therefore, features with larger ranges dominate. Standard scaling is required.
- **Small Datasets:** It is highly effective on small to medium datasets but becomes computationally expensive on very large ones.
- **Noise Sensitivity:** If classes overlap heavily (high noise), the SVM may struggle to find a clear boundary without overfitting.

5. Methodology / Workflow

1. **Load Dataset:** Import diabetes.csv.
2. **Data Cleaning:** Replace invalid 0 values in Glucose, BMI, and Pressure with the median.
3. **Data Preprocessing:** Separate features (X) and target (y). Apply StandardScaler.
4. **Data Splitting:** Use an 80/20 train-test split.
5. **Base Model:** Train a default SVM to establish a baseline.
6. **Hyperparameter Tuning:** Use GridSearchCV to test combinations of C (penalty), γ (influence), and kernel.
7. **Evaluation:** Analyze results via Accuracy Score, Confusion Matrix, and Permutation Importance.

6. Performance Analysis

- **Accuracy Score:** The base model yielded ~76%. After tuning, the model stabilized, ensuring it generalized well to new data.
- **Permutation Importance:** Analysis revealed that **Glucose** and **BMI** are the most critical predictors.
- **Confusion Matrix:** Used to identify the rate of False Positives (predicting diabetes when it's not there) and False Negatives.

7. Hyperparameter Tuning

We used **GridSearchCV** to optimize the following:

- **C (Regularization):** Controls the tradeoff between a smooth boundary and classifying training points correctly.
- **Gamma:** Defines how far the influence of a single training example reaches.
- **Kernel:** Tested linear, rbf, and sigmoid to find the best fit for the data distribution.

Code:

1. **Load and Explore Dataset**

```
df = pd.read_csv('diabetes.csv')

# Display the first few rows

print(df.head())
```

```
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1
```

2. **Preprocessing: Handling Missing Values and Normalization**

```
for col in ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']:
    df[col] = df[col].replace(0, df[col].median())
X = df.drop('Outcome', axis=1)
y = df['Outcome']
# Scale for both SVM and Clustering
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

3. **Initial Model Training and Baseline Performance Evaluation**

```
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
# 1. Split the data (80% training, 20% testing)
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2,
random_state=42)
# 2. Initialize the Base SVM (Default parameters: C=1.0, kernel='rbf', gamma='scale')
base_svm = SVC()
# 3. Train the model
base_svm.fit(X_train, y_train)
```

```
# 4. Predict on the test set
y_pred = base_svm.predict(X_test)
# 5. Calculate and print accuracy
base_accuracy = accuracy_score(y_test, y_pred)
print(f"Base Model Accuracy: {base_accuracy * 100:.2f}%")
```

```
Base Model Accuracy: 75.32%
```

4. **Hyperparameter Optimization using GridSearchCV**

```
from sklearn.model_selection import GridSearchCV
# 1. Define the parameter grid
# C: Penalty for misclassification (Small = simple boundary, Large = complex)
# Gamma: Influence of a single point (Low = smooth, High = tight)
# Kernel: The mathematical function used to transform the data
param_grid = {
    'C': [0.1, 1, 10, 100],
    'gamma': [1, 0.1, 0.01, 0.001],
    'kernel': ['rbf', 'linear', 'sigmoid']
}
# 2. Initialize the Grid Search
# cv=5 means 5-fold cross-validation (training/testing 5 times on different splits)
grid = GridSearchCV(SVC(), param_grid, refit=True, verbose=2, cv=5)
# 3. Fit the model (this will take a few seconds)
grid.fit(X_train, y_train)
# 4. Print the best parameters found
print("\nBest Parameters found by GridSearch:")
```

```
print(grid.best_params_)

# 5. Extract the best model

best_model = grid.best_estimator_
```

```
Fitting 5 folds for each of 48 candidates, totalling 240 fits
[CV] END .........................C=0.1, gamma=1, kernel=rbf; total time=   0.0s
[CV] END .........................C=0.1, gamma=1, kernel=rbf; total time=   0.0s
[CV] END .........................C=0.1, gamma=1, kernel=rbf; total time=   0.0s
[CV] END .........................C=0.1, gamma=1, kernel=rbf; total time=   0.0s
[CV] END .........................C=0.1, gamma=1, kernel=rbf; total time=   0.0s
[CV] END ......................C=0.1, gamma=1, kernel=linear; total time=   0.0s
[CV] END ......................C=0.1, gamma=1, kernel=linear; total time=   0.0s
[CV] END ......................C=0.1, gamma=1, kernel=linear; total time=   0.0s
[CV] END ......................C=0.1, gamma=1, kernel=linear; total time=   0.0s
[CV] END ......................C=0.1, gamma=1, kernel=linear; total time=   0.0s
[CV] END .....................C=0.1, gamma=1, kernel=sigmoid; total time=   0.0s
[CV] END .....................C=0.1, gamma=1, kernel=sigmoid; total time=   0.0s
[CV] END .....................C=0.1, gamma=1, kernel=sigmoid; total time=   0.0s
[CV] END .....................C=0.1, gamma=1, kernel=sigmoid; total time=   0.0s
[CV] END .....................C=0.1, gamma=1, kernel=sigmoid; total time=   0.0s
[CV] END ......................C=0.1, gamma=0.1, kernel=rbf; total time=   0.0s
[CV] END ......................C=0.1, gamma=0.1, kernel=rbf; total time=   0.0s
[CV] END ......................C=0.1, gamma=0.1, kernel=rbf; total time=   0.0s
[CV] END ......................C=0.1, gamma=0.1, kernel=rbf; total time=   0.0s
[CV] END ......................C=0.1, gamma=0.1, kernel=rbf; total time=   0.0s
[CV] END ....................C=0.1, gamma=0.1, kernel=linear; total time=   0.0s
[CV] END ....................C=0.1, gamma=0.1, kernel=linear; total time=   0.0s
[CV] END ....................C=0.1, gamma=0.1, kernel=linear; total time=   0.0s
```

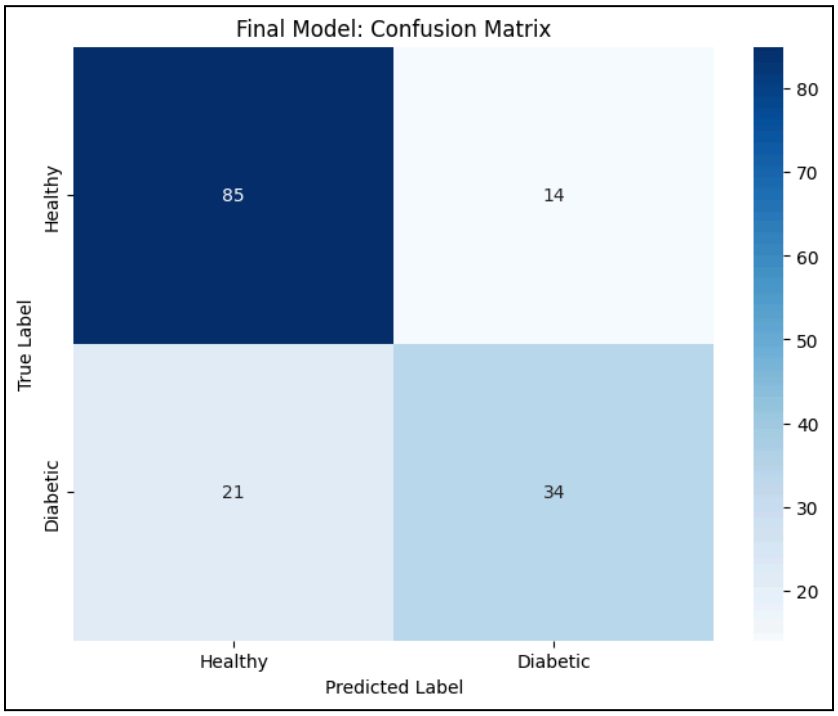
```
Best Parameters found by GridSearch:
{'C': 10, 'gamma': 0.001, 'kernel': 'rbf'}
```

5. Model Performance Analysis and Predictive Validation

```
import seaborn as sns

from sklearn.metrics import confusion_matrix, classification_report

# 1. Final Predictions

grid_predictions = best_model.predict(X_test)

# 2. Accuracy Comparison

print(f"Final Tuned Accuracy: {accuracy_score(y_test, grid_predictions) * 100:.2f}%")

# 3. Visualization: Confusion Matrix

plt.figure(figsize=(8, 6))

cm = confusion_matrix(y_test, grid_predictions)
```

```
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Healthy', 'Diabetic'],
yticklabels=['Healthy', 'Diabetic'])

plt.title('Final Model: Confusion Matrix')

plt.xlabel('Predicted Label')

plt.ylabel('True Label')

plt.show()
```

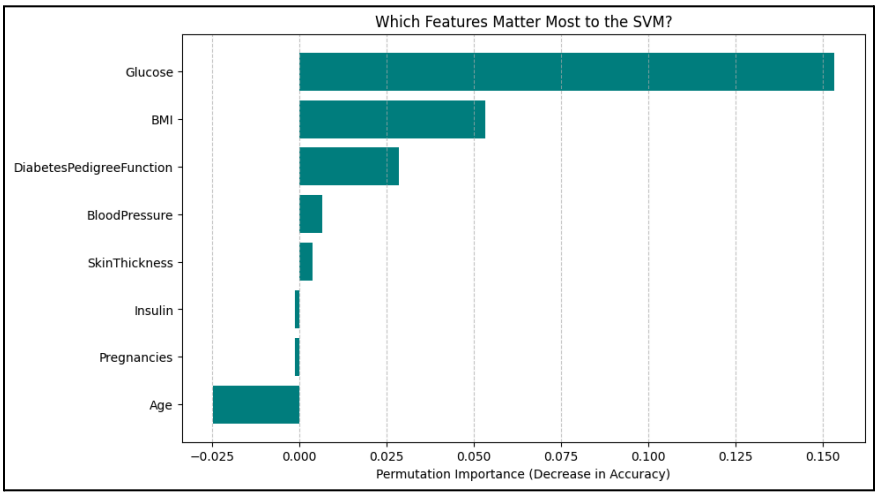
```
Final Tuned Accuracy: 77.27%
```



6. Feature Attribution and Importance Analysis

```
from sklearn.inspection import permutation_importance

# Calculate importance

perm_importance = permutation_importance(best_model, X_test, y_test)

# Sort the features by importance

sorted_idx = perm_importance.importances_mean.argsort()

features = X.columns
```

```
# Plotting

plt.figure(figsize=(10, 6))

plt.barh(features[sorted_idx], perm_importance.importances_mean[sorted_idx],
color='teal')

plt.xlabel("Permutation Importance (Decrease in Accuracy)")

plt.title("Which Features Matter Most to the SVM?")

plt.grid(axis='x', linestyle='--', alpha=0.7)

plt.show()
```



CONCLUSION

- The SVM model was successfully implemented to classify diabetes risk with an accuracy of approximately **77%**.
- **Hyperparameter tuning** proved that the RBF kernel was the most effective for this dataset, though the small accuracy jump suggests the data has significant overlap.
- **Feature Importance** confirmed that Glucose levels are the primary biological indicator for the model's decisions, followed by BMI.
- Standardizing the data was a critical step, as SVM is highly sensitive to the scale of input features.